

Arquitectura de Recomendaciones con Backend de Python y Spring Boot

Esta es una arquitectura común y útil para integrar recomendaciones con IA, en un sistema donde se usan dos backends: uno en Python para la lógica de IA y otro en Spring Boot como fuente de datos. Es una forma flexible de organizar el sistema y puede ser una buena base para seguir desarrollando.

Arquitectura Propuesta: Frontend -> Python Backend -> Spring Boot Backend

Así funciona el flujo:

1. **Frontend:**

- El usuario hace clic en "Ver Recomendaciones".
 - El frontend hace una solicitud HTTP al backend en Python (por ejemplo, `GET /ia/usuarios/{userId}/recomendaciones`).

2. **Backend en Python (encargado de la IA):**

- Recibe la solicitud.
- Llama al backend de Spring Boot para obtener los libros que ha leído el usuario.
- Con esos datos, construye un prompt para enviar a Gemini.
- Envía el prompt a Gemini y devuelve las recomendaciones al frontend.

3. **Backend en Spring Boot:**

- Solo se encarga de exponer datos del usuario.
- Responde con la lista de libros en formato JSON.

Ventajas de esta Arquitectura

- **Separación clara de responsabilidades:** Python gestiona la IA y Spring Boot los datos.

Arquitectura de Recomendaciones con Backend de Python y Spring Boot

- **La lógica de IA está centralizada en un solo sitio (Python):** así es más fácil cambiar el prompt o ajustar cómo se usan los modelos.
- **Spring Boot no necesita saber nada de Gemini:** se mantiene más limpio y enfocado solo en la base de datos.
- **Facilita agregar nuevas funciones relacionadas con IA más adelante.**

Detalles Técnicos a Tener en Cuenta

- **Autenticación:** si los datos son sensibles, se debería usar algún sistema para proteger las llamadas entre Python y Spring Boot.
- **Rendimiento:** al añadir un paso intermedio, puede aumentar un poco la latencia, pero no debería ser un problema para este tipo de uso.
- **Errores:** ambos backends deberían estar preparados para manejar fallos o respuestas inesperadas.

Ejemplo de Código en el Backend de Python

Un ejemplo básico de cómo podría estar hecho en Flask:

```
```python
@app.route('/ia/usuarios/<int:user_id>/recomendaciones', methods=['GET'])
def obtener_recomendaciones_para_usuario(user_id):
 springboot_url = f"http://localhost:8080/api/usuarios/{user_id}/libros-leidos"
 response = requests.get(springboot_url)
 libros_leidos = response.json()

 prompt = "El usuario ha leído:
"
```

## Arquitectura de Recomendaciones con Backend de Python y Spring Boot

```
for libro in libros_leidos:
 prompt += f"- {libro['titulo']} de {libro['autor']}"
 ""

 prompt += "Recomiéndale 5 libros similares, explica por qué."

 response_from_gemini = gemini_model.generate_content(prompt)
 recomendaciones = response_from_gemini.text.strip()
 return jsonify({"recomendaciones": recomendaciones})
'''

```

Esta arquitectura permite tener la lógica de IA separada del resto del sistema y da bastante libertad para experimentar con recomendaciones sin tener que modificar el backend que maneja los datos.